

Accelerating SystemVerilog Assertions Proof Using LLMs and Machine Learning

July 10, 2026

1 Introduction

Formal Property Verification (FPV) establishes that a SystemVerilog design satisfies an assertion written in SystemVerilog Assertions (SVA), but its cost grows quickly with the size of the design’s state space. Two complementary strategies are commonly used by verification engineers to mitigate this cost:

- **Hierarchical lemma generation:** produce a directed tree of assertions rooted at the target, with helper assertions as descendants. Each assertion should be easier to verify once its child properties are assumed to be true, decomposing a hard proof into a sequence of tractable ones.
- **State-space reduction:** introduce cutpoints, abstractions, and blackboxing to shrink the space the verifier must explore.

We propose to train an LLM that accelerates FPV by interacting with the verifier in a closed loop: at each step it adds helper assertions, cutpoints, abstractions, and blackboxing, observes the verifier’s feedback, and refines its next move accordingly.

This mirrors mathematical proof in Lean, yet SVA is more forgiving: there is no need to construct a full derivation from axioms, nor to formally derive each lemma from its dependencies. Verification also offers a far richer training signal than the binary success/failure of a Lean proof, since the time taken to discharge an assertion provides a graded measure of proof quality.

These advantages suggest that generating helper assertions with an LLM is at least as promising as LLM-based proof generation in Lean, an approach that has already proven highly effective, especially in combination with reinforcement learning. AlphaProof reached silver-medal performance on IMO problems [7], while the DeepSeek-Prover series advanced Lean proving via RL from proof-assistant feedback [39] and, more recently, via RL for subgoal decomposition [30]—the latter being closely analogous to our helper-assertion generation.

2 Model

This work can use a standard LLM architecture; we plan to start from an open-source pretrained LLM. The model should have agentic and reasoning capabilities, and a context window larger than 32k tokens since the design files can be quite large.

Candidates:

- **Qwen3.6-36B-A3B** (36B total, $\sim$ 3B active) — MoE; primary choice for the full PPO run.
- **Qwen3.6-27B** (28B, dense) — simpler to fine-tune; used for environment and reward bring-up.

We plan to validate the environment on the dense Qwen3.6-27B, then scale to the MoE Qwen3.6-36B-A3B for the full run.

3 Fine-Tuning

Reinforcement learning explores efficiently only from a competent starting policy. We therefore fine-tune the base LLM on a domain corpus to give it fluency in SystemVerilog, SVA, and the assume-guarantee contract structure before the RL stage (Section 4).

3.1 Data Sources

No single public dataset labels assume-guarantee pairs as such: existing SVA corpora emphasize **assert property** generation but rarely include the paired **assume property** statements that define environment constraints. We therefore compose the corpus from complementary sources.

Source	Purpose	Verified labels	Used for
HierSVA [2]	Hierarchical contracts	Yes	SFT/RL
OpenTitan [17]	Assume-guarantee structure	Yes	SFT/RL
HWSec-UNC [1]	Security properties	Yes	SFT/RL
riscv-formal [35]	Compositional verification	Yes	SFT/RL
VERT [22]	SVA syntax breadth	Mixed	SFT
Verixa [36]	Verification context	Mixed	SFT
NVIDIA CVDP [25]	Specification and RTL	Mixed	SFT
VerilogEval [15]	English and RTL	No	SFT

Table 1: Fine-tuning and reinforcement-learning data sources.

3.2 Staged Fine-Tuning

We fine-tune on this combined corpus in two stages: the first covers SystemVerilog and SVA syntax and temporal operators; the second covers the assume-guarantee contract structure. This grounding gives the model domain competence before reinforcement learning refines its assume-guarantee proof strategy.

4 Reinforcement Learning

We train the fine-tuned LLM with PPO (Proximal Policy Optimization) [32] and Generalized Advantage Estimation (GAE) [31].

4.1 The environment

Initialization. The environment starts with a SystemVerilog design and an assertion to prove.

Action. The agent’s action is a directed tree of assertions rooted at the target, with helper assertions as descendants, together with cutpoint, abstraction, and blackboxing commands.

Environment mechanics. Given an action, the environment applies the cutpoint, abstraction, and blackboxing commands to the design. It then verifies the assertions in the tree in topological order; when proving an assertion, it assumes its descendants properties are true.

Observation. The agent observes the design and the target property. If the prover fails, the agent also observes which assertions failed and why (e.g., counterexample or timeout).

Termination. The environment terminates when the target assertion is proven or when a maximum number of steps is reached.

4.2 Prover flow

The environment requires a prover that parses SVA, discharges assertions unattended at scale, and supports the cutpoint, abstraction, and blackboxing commands of the action space. Candidate flows:

- **SymbiYosys + Verific (Tabby CAD Suite)** [43]: Yosys-based flow with the industry-standard Verific frontend. Full IEEE 1800 SVA support; cutpoints and blackboxing map directly to Yosys passes; engines include BMC, k-induction, and IC3 via SMT solvers and ABC. Proprietary but free for academic use.
- **SymbiYosys (open-source Yosys frontend)** [43]: same flow without Verific. Fully open source, but parses only a subset of SVA (immediate and simple concurrent assertions), which would exclude the temporally deep properties in our corpus.
- **EBMC** [24]: word-level bounded model checker with k-induction from the CPROVER family. Native SystemVerilog frontend with broader SVA coverage than open-source Yosys; free and dependency-light, so trivial to deploy on a cluster, but weaker engine variety and no built-in cutpoint support.
- **Commercial tools (e.g., JasperGold, VC Formal)**: strongest engines and full SVA, but license-server seats constrain the parallel, unattended proof workers our RL loop requires (Section 7).

We use SymbiYosys with the Verific frontend as the primary flow—it is the only free option that combines full SVA parsing with unattended operation and native cutpoint/blackbox support—and EBMC as a fallback for designs outside the Verific flow.

4.3 Reward design

Each step receives a large positive reward when the target assertion is proved. Separately, every assertion in the tree incurs a penalty based on the time the engine spent checking it and on whether it was left unproven.

4.4 Designs and assertions for the environment

Each instance pairs a SystemVerilog design with a target assertion that must satisfy **ground-truth provability**—the target is known to hold, so failures are attributable to the agent rather than to a false property (we use only formally verified SVA, never generated assertions of unknown status).

Sources. We reuse the verified-SVA subset of the fine-tuning corpus (Section 3.1): HierSVA [2], OpenTitan [17], riscv-formal [35], and HWSec-UNC verification-benchmarks [1].

Splitting and leakage control. Instances are partitioned by design (not assertion) into training and held-out sets, so no module seen during RL appears at test time. Designs shared with the fine-tuning corpus stay in the training split, keeping the held-out set a clean measure of generalization to unseen designs.

4.5 Curriculum learning

Difficulty grading. Each pair carries a static difficulty proxy—cone-of-influence state-bit count, hierarchy depth of the supporting module tree, and temporal depth of the property (nesting of `##`, `|->`, and unbounded operators)—calibrated against baseline single-shot proof time and timeout rate and binned into tiers consumed by the curriculum.

We start the LLM on simple designs and assertions with low timeouts, and gradually increase the complexity of the designs and assertions, as well as the timeouts, as the LLM improves.

5 Related Work and Novelty

5.1 Formal Verification of Hardware

The dominant scalability bottleneck in industrial FPV is state-space explosion on deep, control-heavy properties. Engineers manually decompose end-to-end properties into helper lemmas, add cutpoints, black-box submodules, and write assume-guarantee contracts. This human effort is exactly the workflow we propose to automate.

5.2 LLMs for Hardware Design and Verification

Recent work has applied LLMs to RTL and verification artifacts, but predominantly to *generation* rather than *proof*:

- **RTL/code generation:** VerilogEval [15], RTLCoder [16], ChipNeMo [14], BetterV [26].

- **Assertion generation from English/RTL:** AssertLLM [5], ChIRAAG [18], SpecLLM [12], and NSPG [21].
- **Testbench and stimulus generation:** AutoBench [29], ChatEDA [9].

These works treat the LLM as a *producer of properties to check*, leaving the proof of a given property to the engineer.

5.3 LLM-Driven Lemma Synthesis for Hardware Proofs

The closest prior work is the neurosymbolic lemma-mining framework of Peled et al. [27], which does close the loop with a model checker: an LLM proposes candidate invariants and a sound symbolic procedure (FINDINDSTRENGTH) sifts them for a subset that forms an inductive strengthening of the target property, discharged with k -induction and BMC via EBMC. They compare a *non-agentic* single few-shot prompt against an *agentic* loop that re-prompts with verifier feedback, evaluate six SOTA LLMs on 109 SystemVerilog (D, φ) verification tasks, and report that at least one prompting setup produces a provably correct inductive argument for 90% of tasks, solving instances on which JasperGold, VC-Formal, and rIC3 time out.

Our work shares this neurosymbolic loop—LLM proposes, sound oracle verifies, successful trajectories are reused—but differs in three respects. First, where [27] relies purely on few-shot prompting of frozen models with a manually curated chain-of-thought pool, we *fine-tune* a domain model (Section 3) and refine its proof strategy with *reinforcement learning* (Section 4), removing the dependence on a hand-built example pool and letting the policy improve from its own verified trajectories. Second, we target the full assume-guarantee workflow—contracts at module boundaries, cutpoint insertion, and engine selection—rather than monolithic inductive strengthening of a single property. Third, we address hierarchical, compositional decomposition rather than flat invariant synthesis.

5.4 ML / LLMs for Theorem Proving

The closest methodological neighbours are neural theorem provers in interactive proof assistants: GPT-f and the OpenAI Lean work [28], Llemma [3], ReProver/LeanDojo [41], Lean-STaR [13], DeepSeek-Prover (v1/v1.5/v2) [38, 39, 30] with expert iteration and RL against the Lean kernel, and AlphaProof [7]. The shared recipe—LLM policy proposes proof steps, a sound oracle verifies, successful trajectories feed expert iteration—transfers naturally to FPV. The crucial structural difference is that SVA proofs do not require constructing a derivation from axioms: a *single* well-chosen lemma whose conjunction with trivial facts implies the goal and is dischargeable by BMC/IC3 already closes the obligation. The branching factor of useful actions is therefore much smaller than in Lean, and partial credit is much easier to assign.

5.5 Loop and Class Invariant Generation

A related line generates loop or class invariants for software with LLMs, e.g. Microsoft’s loop-invariant experiments [23], ClassInvGen [34], and the studies of Janßen et al. [10], Wu et

al. (Lemur) [37], and Yao et al. [42]. These tasks are conceptually different: loop invariants are local to a loop body and need a state predicate preserved by one iteration, whereas inductive arguments for hardware proofs are global, may involve temporal operators, and capture facts about all reachable states of a transition system. Peled et al. [27] adapt 47 such C loop-invariant examples into SystemVerilog finite-state machines, illustrating the gap between the two settings.

5.6 Algorithm / Engine Selection and Portfolio Solving

The “select the best engine” sub-task aligns with SATzilla [40], MachSMT [33], and per-instance algorithm-selection literature for SAT/SMT/CHC.

5.7 Cutpoints and Abstractions

Automatic abstraction for hardware model checking spans localization abstraction [11], CEGAR [4], predicate abstraction [8], proof-based abstraction [20], and signal/term abstraction in word-level model checkers (AVR [6], Pono [19]). Manual cutpoint insertion remains a notoriously difficult task.

5.8 Hierarchical Lemma Generation

Existing LLM-based lemma synthesis for hardware operates on a single property and a flat transition system [27]. Industrial proofs instead decompose across the module hierarchy, proving guarantees of submodules under environment assumptions and composing them into system-level properties. We target this hierarchical, compositional setting directly, mirroring the depth-structured assertions in datasets such as HierSVA (Section 3).

5.9 SystemVerilog and SVA Benchmarks for LLMs

Benchmarks for LLM-based hardware verification remain small and generation-oriented. Peled et al. [27] assemble 109 (D, φ) proof tasks from 57 SystemVerilog designs (drawn from VCEGAR, NeuralMC, VIS, v2c, and parametrized FIFO/LRG/CAM designs), the only public benchmark targeting hardware *proof* rather than property generation. For assertion generation, VERT [22] provides $\sim 20,000$ SVA samples and the Verixa corpus [36] 105,340 mixed verification samples, while NVIDIA CVDP [25] pairs RTL with specifications and testbenches. None of these label assume-guarantee contracts or supply proof trajectories, motivating the corpus construction and augmentation of Section 3.

6 Experiments

7 Cost Estimate

We estimate the compute cost of the full pipeline — staged fine-tuning (Section 3), environment bring-up, and the PPO run (Section 4) — against Verda spot prices.¹

7.1 Assumptions

- **Hardware.** A single B200 SXM6 (\$2.14/GPU-hour spot) for all GPU work; proof workers on CPU (\$0.0025/core-hour spot).
- **Model.** Qwen3.6-36B-A3B (MoE, ~ 3 B active), served in FP8 (~ 36 GB of weights).
- **Fine-tuning is parameter-efficient (LoRA).** The combined corpus is ~ 130 k samples and the two curriculum stages run ~ 3 epochs in total, for ~ 0.78 B training tokens.
- **Rollout generation.** $\sim 4,000$ generated tokens per environment step at an aggregate FP8 decode throughput of $\sim 24,000$ tokens/s; PPO policy and value-function updates add $\sim 25\%$ on top of generation GPU-time.
- **Proofs.** One formal proof per environment step, ~ 30 s on a single CPU core.

7.2 Total

Stage	Low	Central	High
Fine-tuning (LoRA, 2 stages)	\$120	\$120	\$120
Environment bring-up	\$15	\$15	\$15
RL run	\$18	\$58	\$116
Total	\$153	\$193	\$251

Table 2: End-to-end compute cost at Verda spot prices. The RL cases use 0.125 M, 0.4 M, and 0.8 M environment steps.

These figures assume a clean run; budgeting a $1.5\times$ contingency for failed runs and iteration gives a planning range of $\sim \$230$ – $\$377$. The estimate is most sensitive to total environment steps and tokens-per-step, which scale the RL GPU term linearly.

¹Verda spot pricing, retrieved 2026-06: B200 SXM6 at \$2.14/GPU-hour and CPU at \$0.0025/core-hour (<https://verda.com/products>).

8 Conclusions

References

- [1] Hammad Ahmad, Yu Huang, and Westley Weimer. Security properties for open-source hardware designs. *arXiv preprint arXiv:2412.08769*, 2024.
- [2] Anonymous. HierSVA: A data synthesis pipeline, dataset, and benchmark for hierarchical SystemVerilog assertions. Hugging Face dataset <https://huggingface.co/datasets/AnonymousHierSVA/HierSVA>, 2026.
- [3] Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q. Jiang, Jia Deng, Stella Biderman, and Sean Welleck. Llemma: An open language model for mathematics. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [4] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV)*, 2000.
- [5] Wenji Fang, Mengming Wang, Yao Lu, Shang Liu, Hongce Zhang, and Zhiyao Xie. AssertLLM: Generating hardware verification assertions from design specifications via multi-LLMs. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2025.
- [6] Aman Goel and Karem Sakallah. AVR: Abstractly verifying reachability. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2020.
- [7] Google DeepMind AlphaProof and AlphaGeometry Teams. AI achieves silver-medal standard solving international mathematical olympiad problems. Google DeepMind Blog, 2024.
- [8] Susanne Graf and Hassen Saïdi. Construction of abstract state graphs with PVS. In *Computer Aided Verification (CAV)*, 1997.
- [9] Zhuolun He, Haoyuan Wu, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. ChatEDA: A large language model powered autonomous agent for EDA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [10] Christian Janßen, Cedric Richter, and Heike Wehrheim. Can ChatGPT support software verification? In *Fundamental Approaches to Software Engineering (FASE)*, 2024.
- [11] Robert P. Kurshan. *Computer-Aided Verification of Coordinating Processes: The Automata-Theoretic Approach*. Princeton University Press, 1994.
- [12] Mengming Li, Wenji Fang, Qijun Zhang, and Zhiyao Xie. SpecLLM: Exploring generation and review of VLSI design specification with large language model. *arXiv preprint arXiv:2401.13266*, 2024.

- [13] Haohan Lin, Zhiqing Sun, Yiming Yang, and Sean Welleck. Lean-STaR: Learning to interleave thinking and proving. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.
- [14] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, Chris Cheng, Nathaniel Pinckney, Rongjian Liang, Jonah Alben, Himyanshu Anand, Sanmitra Banerjee, Ismet Bayraktaroglu, et al. ChipNeMo: Domain-adapted LLMs for chip design. *arXiv preprint arXiv:2311.00176*, 2023.
- [15] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [16] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. RTL-Coder: Outperforming GPT-3.5 in design RTL generation with our open-source dataset and lightweight solution. *arXiv preprint arXiv:2312.08617*, 2024.
- [17] lowRISC. OpenTitan: Open source silicon root of trust. <https://github.com/lowRISC/opentitan>, 2024.
- [18] Bhabesh Mali, Karthik Maddala, Sweeya Reddy, Vatsal Gupta, Chandan Karri, and Subhasis Roy. ChIRAAG: ChatGPT informed rapid and automated assertion generation. *arXiv preprint arXiv:2402.00093*, 2024.
- [19] Makai Mann, Ahmed Irfan, Florian Lonsing, Yahan Yang, Hongce Zhang, Kristopher Brown, Aarti Gupta, and Clark Barrett. Pono: A flexible and extensible SMT-based model checker. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2021.
- [20] Kenneth L. McMillan and Nina Amla. Automatic abstraction without counterexamples. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2003.
- [21] Xingyu Meng, Amisha Srivastava, Ayushi Arunachalam, Avik Ray, Pedro Henrique Silva, Rafail Psiakis, Yiorgos Makris, and Kanad Basu. NSPG: Natural language processing-based security property generator for hardware security assurance. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [22] Anand Menon, Vatsal Gupta, Chandan Karri, Tajana Rosing, and Kanad Basu. Enhancing large language models for hardware verification: A novel SystemVerilog assertion dataset. *arXiv preprint arXiv:2503.08923*, 2025.
- [23] Microsoft. Loop invariant generation with LLMs. <https://github.com/microsoft/loop-invariant-gen-experiments>, 2024.
- [24] Rajdeep Mukherjee, Peter Schrammel, Daniel Kroening, and Tom Melham. Unbounded safety verification for hardware using software analyzers. In *Design, Automation & Test in Europe (DATE)*, 2016.

- [25] NVIDIA. CVDP: Comprehensive Verilog design problems benchmark dataset. Hugging Face dataset <https://huggingface.co/datasets/nvidia/cvdp-benchmark-dataset>, 2025.
- [26] Zehua Pei, Hui-Ling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. BetterV: Controlled Verilog generation with discriminative guidance. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [27] Romy Peled, Daniel Kroening, Michael Tautschnig, and Yakir Vizel. Large lemma miners: Can LLMs do induction proofs for hardware? *arXiv preprint arXiv:2511.02521*, 2026.
- [28] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- [29] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. AutoBench: Automatic testbench generation and evaluation using LLMs for HDL design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, 2024.
- [30] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.
- [31] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016.
- [32] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [33] Joseph Scott, Aina Niemetz, Mathias Preiner, Saeed Nejati, and Vijay Ganesh. MachSMT: A machine learning-based algorithm selector for SMT solvers. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2021.
- [34] Chuyue Sun, Viraj Agashe, Saikat Chakraborty, Jubi Taneja, Clark W. Barrett, David L. Dill, Xiaokang Qiu, and Shuvendu K. Lahiri. ClassInvGen: Class invariant synthesis using large language models. *arXiv preprint arXiv:2502.18917*, 2025.
- [35] SymbioticEDA. riscv-formal: RISC-V formal verification framework. <https://github.com/SymbioticEDA/riscv-formal>, 2024.
- [36] Vyoma Labs. Verixa dataset: Hardware verification, SVA, UVM, and formal verification corpus. Hugging Face dataset <https://huggingface.co/datasets/vyomaalabs/verixa-dataset>, 2025.

- [37] Haoze Wu, Clark W. Barrett, and Nina Narodytska. Lemur: Integrating large language models in automated program verification. In *International Conference on Learning Representations (ICLR)*, 2024.
- [38] Huajian Xin, Daya Guo, Zhihong Shao, Zhizhou Ren, Qihao Zhu, Bo Liu, Chong Ruan, Wenda Li, and Xiaodan Liang. DeepSeek-Prover: Advancing theorem proving in LLMs through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*, 2024.
- [39] Huajian Xin, Z. Z. Ren, Junxiao Song, Zhihong Shao, Wanjia Zhao, Haocheng Wang, Bo Liu, Liyue Zhang, Xuan Lu, Qiushi Du, et al. DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search. *arXiv preprint arXiv:2408.08152*, 2024.
- [40] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- [41] Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.
- [42] Jianan Yao, Ziqiao Zhou, Weiteng Chen, and Weidong Cui. Leveraging large language models for automated proof synthesis in Rust. *arXiv preprint arXiv:2311.03739*, 2023.
- [43] YosysHQ. SymbiYosys (sby): Front-end for Yosys-based formal verification flows. <https://github.com/YosysHQ/sby>, 2024.